AI Engineering and Claude Architect · Part II, Retrieval

Vector Search, Lesson 5

Design a Multi-Tenant Vector Search Platform

Five thousand customers share your retrieval platform. One holds 22 million chunks, and the median holds under 9,000. A single query from the wrong tenant must never see another's data, and the largest must never slow down the smallest. Design it.

Module	AI04 of 13
Lesson	5 of 5, including the module revision. Design problem
Prerequisites	AI04 L1–L4, HLD M07–M08 and M21A
Exam relevance	Out of scope. A strong senior interview topic
Reading time	~45 minutes

Contents

1. Design a Multi-Tenant Vector Search Platform
2. Module Revision

Lesson 5 of 5 · Vector Databases and Search · Design problem

Design a Multi-Tenant Vector Search Platform

Key idea

Tenant sizes are wildly uneven, so one tenancy model can't fit all: share storage among the many small tenants, and give the few large ones dedicated indexes. Enforce isolation in more than one layer, because a filter you forgot is a data breach. **Size by memory first, treat the index as rebuildable, and keep exact results as your measure of quality.**

> *Before: everything in this module. Teaches: combining embedding operations, filtering, hybrid search and store choice into one defensible design. Used by: AI05 L7 (knowledge assistant), AI10 L6 (multi-tenancy).*

All numbers below are stated assumptions for this exercise, and this isn't a description of any real company's system.

1. Scope

Questions to ask before you draw anything, and what each answer changes.

Question	Assumed answer	What it changes
How many tenants, and how uneven?	5,000, very uneven	One model won't fit all, so tiers
Total chunks?	200 million	Needs compression and sharding
Cross-tenant search?	Never	Every query can route to one place
Isolation requirement?	No data crossing, audited	Enforced in several layers
Latency target?	Retrieval p99 under 200 ms	Limits reranking and fan-out
Freshness?	New documents searchable within 5 minutes	Async ingestion is acceptable
Deletion?	A document within 24 hours, and a tenant within 30 days	Drives storage design
Hybrid search?	Yes, since users paste error codes	Two indexes
Dedicated infrastructure for regulated tenants?	For a few	Same mechanism as large tenants

2. Requirements

Functional	Non-functional	Out of scope
Ingest and chunk documents	Retrieval p99 under 200 ms at peak	LLM answer generation (AI05 L7)
Hybrid search, returning text and sources	99.9% availability for search	Document parsing details
Per-tenant model version	Measured recall@10 of at least 0.95	Billing
Delete documents and tenants	No cross-tenant reads, proven by tests	Cross-tenant analytics
Change embedding model without downtime	One tenant can't degrade another	

3. Estimation

Assumptions: 200 million chunks, 1024 dimensions, Zipf-distributed tenant sizes, and 128 GB nodes with 70% usable for the index. The Zipf shape is **an assumption to validate** against real data. Real output from `code/ai04-vector/TenantPlan.java`:

```
Largest tenant: 21,991,292 chunks. Median tenant: 8,793. Smallest: 4,398
threshold    100,000:   219 dedicated tenants hold 65.6% of data, 4,781 share the rest
threshold  1,000,000:    21 dedicated tenants hold 40.1% of data, 4,979 share the rest
threshold 10,000,000:     2 dedicated tenants hold 16.5% of data, 4,998 share the rest
Vectors: float32 819 GB, int8 205 GB, binary 26 GB, graph 53 GB
float32 + HNSW                         872 GB in RAM   10 nodes   20 with 2 copies
int8 + HNSW                            258 GB in RAM    3 nodes    6 with 2 copies
binary + HNSW, floats on disk           78 GB in RAM    1 nodes    2 with 2 copies
Ingest 500,000 chunks/day = 5.8/s on average
Re-embed all at    500 chunks/s: 4.6 days
Re-embed all at  2,000 chunks/s: 1.2 days
Re-embed all at 10,000 chunks/s: 0.2 days
```

Number	What it means
Largest tenant is 2,500× the median	Fairness and sizing can't be uniform
21 tenants hold 40% of the data	A small set of tenants justifies special handling
872 GB in RAM as float32	**20 nodes, of which most of the cost is raw vectors**
258 GB as int8	**6 nodes**, about 70% cheaper, at the recall cost AI04 L2 measured
Binary looks like 2 nodes	**Don't assume it.** AI04 L2 measured poor recall on clustered data
5.8 chunks/s average ingest	Ingest isn't the bottleneck. Re-embedding is
1 to 5 days to re-embed	Model migrations must run with search live

Throughput needs a benchmark. Suppose your test shows 400 searches per second per replica at the target recall, a **placeholder** for your measurement. A peak of 2,000 queries per second then needs 5 replicas. Memory needs 3 shards, so 3 shards with 2 copies gives 6 replicas, which fits. **Note that routing is what makes this work**: if each query fanned out to all 3 shards, you'd do 6,000 shard searches per second instead of 2,000, which is more than those 6 replicas can serve.

4. API

```
POST /v1/tenants/{tenantId}/documents
{ "documentId": "doc-77", "source": "s3://...", "metadata": {"lang": "en"} }
→ 202 Accepted { "ingestId": "ing-123" }

POST /v1/tenants/{tenantId}/search
{ "query": "ERR-4012 during capture", "k": 20,
  "filter": {"lang": "en"}, "mode": "hybrid" }
→ 200 { "results": [{"chunkId": "...", "text": "...", "source": "...", "score": 0.031}],
        "modelVersion": "voyage-4@1024", "degraded": false }

DELETE /v1/tenants/{tenantId}/documents/{documentId}
→ 202 Accepted
```

Design choice	Why
Tenant from the authenticated token, checked against the path	The caller can't simply name another tenant
`202` for ingest	Embedding is asynchronous
`degraded` flag	Callers know when reranking or vectors were skipped
`source` in results	Provenance for citations (AI05 L4)

5. Data model

Store	Holds	Why there
Tenant catalog (Postgres)	Tier, shard location, model version, quotas	The router's source of truth
Chunk store (Postgres or object storage)	Chunk text, source, hash, full float32 vector	Durable. Rebuilds indexes
Vector index	Compact vector, chunk ID, filter fields	Fast search, and rebuildable
Keyword index	Text terms per chunk	Exact matches

```
-- Tenant catalog: the router reads this.
CREATE TABLE tenants (
    tenant_id     text PRIMARY KEY,
    tier          text NOT NULL,  -- 'shared' or 'dedicated'
    index_target  text NOT NULL,  -- which shard or index
    model_version text NOT NULL,
    query_quota   int  NOT NULL
);
```

6. High-level design

Version 1: one shared index, with a filter

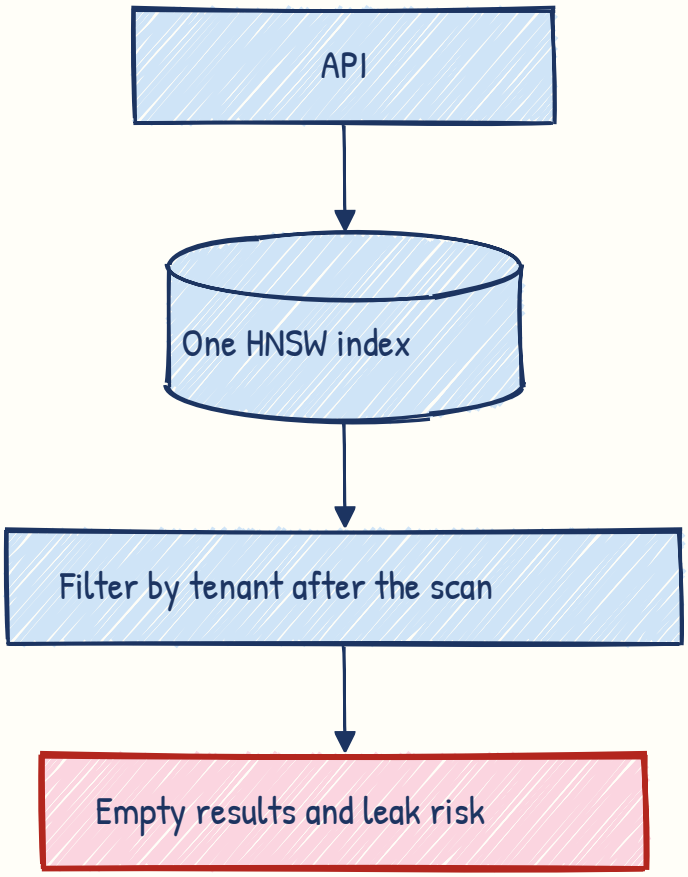


Figure 1. Baseline. One index with a tenant filter looks simple, but correlated filters return nothing, one missing WHERE clause is a breach, and the largest tenant dominates.

AI04 L2 measured this: a tenant owning 10% of the data got empty results on 88% of queries. **The approach fails on both quality and safety.**

Version 2: one index per tenant

Isolation and quality are excellent. **New problem: 5,000 indexes, of which most hold 9,000 vectors or fewer, each with their own build, memory and failure handling.** Qdrant's documentation says that creating a separate collection for each tenant "is rarely the most efficient approach", and Qdrant Cloud caps clusters at 1,000 collections by default.

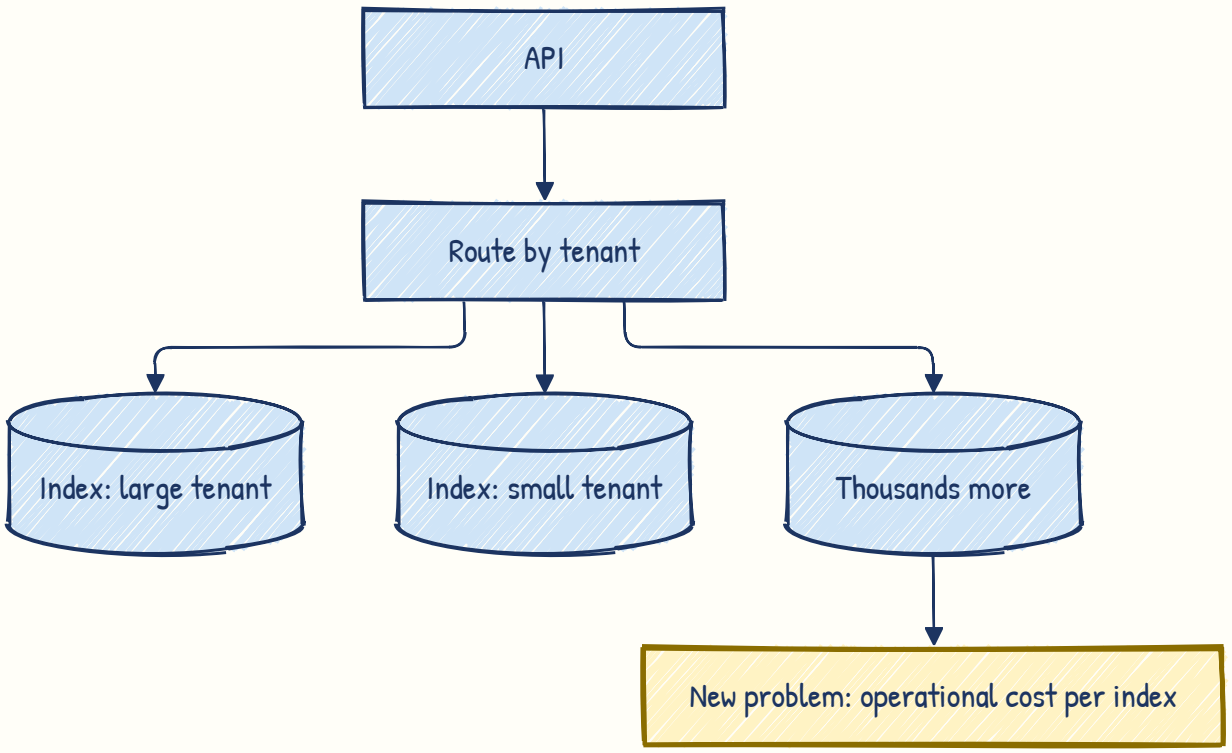


Figure 2. Improved. Isolation is excellent, but thousands of tiny indexes waste memory and operational effort.

Version 3: tiered tenancy

Most tenants share partitioned storage that's organized by tenant. Tenants above a threshold, or with regulatory needs, get dedicated indexes. Qdrant documents this exact shape as **tiered multitenancy**, with small tenants in a shared shard and large ones promoted. Its suggested promotion point differs from our 1 million threshold, so test yours.

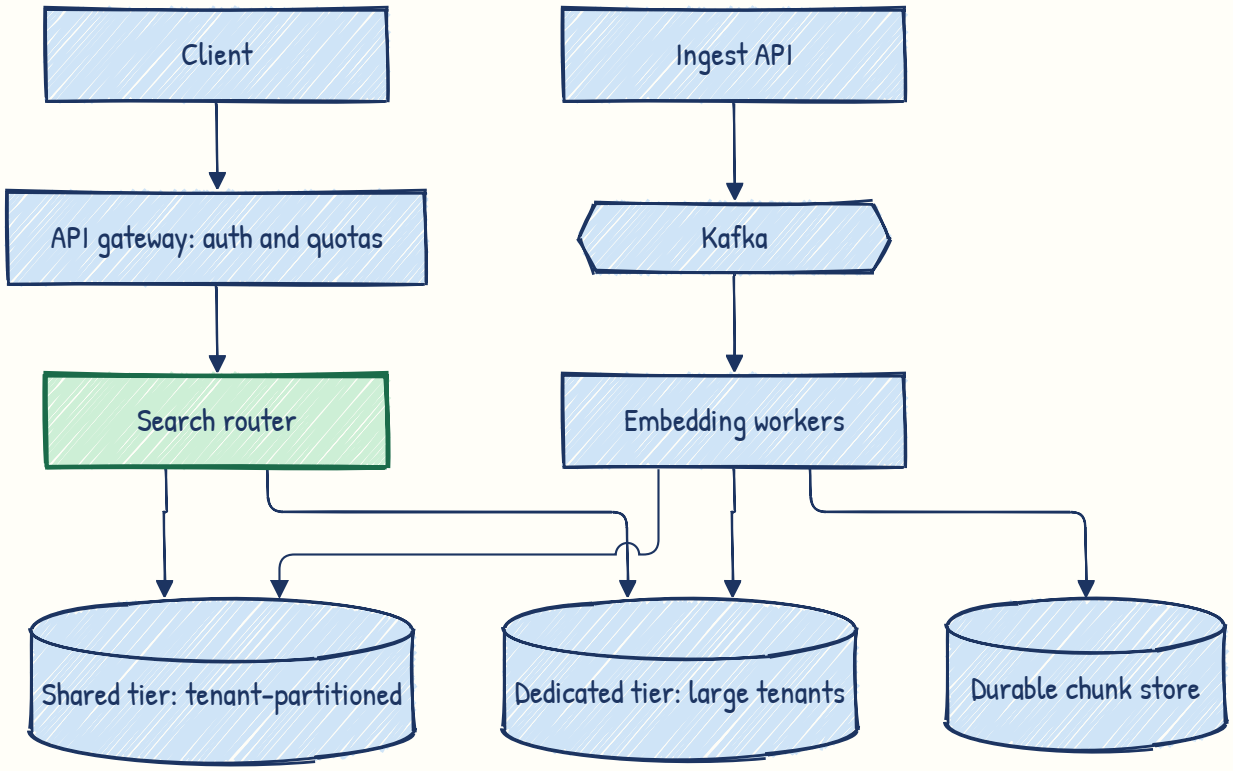


Figure 3. Production. The router reads the catalog and sends each query to exactly one place. Ingestion and search stay separate, and the durable chunk store can rebuild any index.

Component	Why it exists	How it communicates
API gateway	Authenticates, applies per-tenant quotas	HTTPS
Search router	Picks the single target from the catalog, with a cached copy	gRPC to shards
Shared tier	Efficient for thousands of small tenants	Tenant-partitioned search
Dedicated tier	Isolates large or regulated tenants	Own index
Kafka and workers	Absorb ingest, retry safely	Partitioned by tenant
Durable chunk store	Rebuilds indexes, feeds migrations	Written before indexing

The hot path

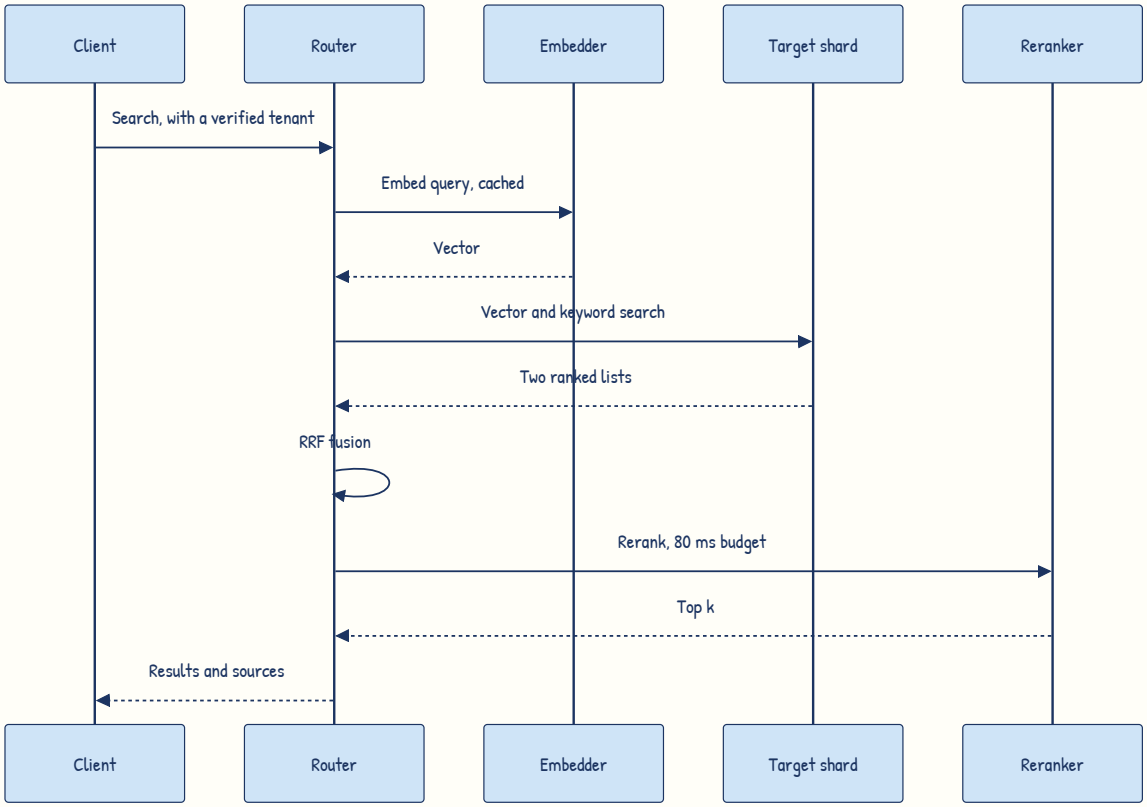


Figure 4. A search. Retrieval runs in parallel with time budgets. Reranking is skipped rather than allowed to breach the latency target.

7. Deep dives

Deep dive 1: Three layers of isolation

A single `WHERE tenant_id = ?` is one forgotten clause from a breach. **Assume any one layer can fail.**

Layer	Mechanism	What it catches
Routing	Tenant comes from the token, not the request body	A client asking for another tenant
Storage	Partition or namespace, or row-level security	A code path missing a filter
Testing	An automated test that attempts cross-tenant reads on every release	Regressions in either layer

The caches matter too. **Include the tenant in every cache key.** A query embedding is safe to share, since it's derived only from the query text. A cached search **result** isn't, and a shared result cache is a classic way to leak data.

One subtle risk: a shared embedding cache can reveal that some tenant searched the same text, through timing. For regulated tenants, use separate caches.

Deep dive 2: Noisy neighbors

Noise source	Control
Query spikes	Per-tenant token bucket at the gateway (HLD M11)
Bulk ingest	Per-tenant Kafka quota, with a low-priority bulk topic
Expensive filters	Limit `k` and filter complexity, with query timeouts
A rapidly growing tenant	Promote it before it hurts neighbors

Deep dive 3: Promoting a tenant with no downtime

Promotion resembles a model migration: build beside, then switch.

Step	Action
1	Detect: the tenant crosses a size or query threshold
2	Build a dedicated index from the durable chunk store
3	Double-write new chunks to both places
4	Compare results on sampled queries
5	Flip `index_target` in the catalog
6	Purge the tenant from the shared tier, after a grace period

Qdrant's documentation describes a shard-transfer form of this, which supports reads and writes during promotion. The steps above work with any store.

Deep dive 4: Memory and recall

Choose **int8 with full vectors on disk**, since that gives a defensible 4× saving, with a two-stage rerank when the first stage's recall isn't enough. Don't commit to binary without measuring it on real embeddings, as AI04 L2's result warns. Run the sweep per embedding model, and track recall per tier.

Deep dive 5: Deletion

Delete type	Approach	Why
A tenant in a dedicated index	Drop the index	Immediate
A tenant in a namespace or partition	Drop it	Pinecone calls namespace deletion "lightweight and almost instant"
A document	Tombstone at once, so that it's filtered at read time, then remove from indexes	HNSW deletes are tricky (HLD M21A)
Backups	Define retention, and record it	Otherwise "deleted" data persists

Deletion is a compliance feature, so make it testable. Record who requested it, when it was tombstoned, and when it was purged.

Deep dive 6: Changing the embedding model

With 200 million chunks and 1–5 days to re-embed, migrate **tenant by tenant**, since the catalog already stores a model version for each. Each tenant is consistent at all times: its queries use its version. Run large tenants off-peak, with a capped bulk quota. This is why the model version sits in the catalog, rather than being a global setting.

8. Scaling and bottlenecks

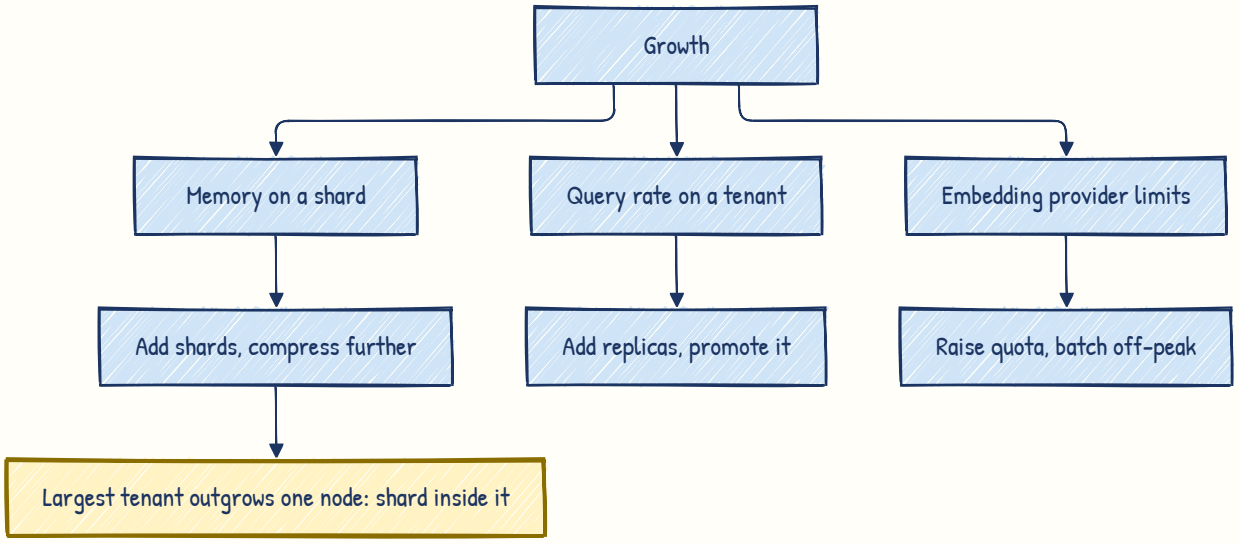


Figure 5. Where it breaks next. Each limit has a different fix, and a large tenant reaching its node's capacity is what finally forces sharding inside a tenant.

Bottleneck	Signal	Response
Shard memory	Index near RAM limit	Rebalance, add shards
A hot tenant	One tenant's share of queries rising	Promote, or add replicas
A tenant bigger than a node	Largest tenant near a node's limit	Shard within the tenant, accepting fan-out for it only
Index builds	Builds take too long	Parallel builds, and build on spare nodes
Router catalog	Lookups on each request	Cached copy, with versioned updates

At our numbers, the largest tenant needs about 28 GB at int8 with its graph, so it fits on one node with plenty of room. Fan-out inside a tenant is a later problem.

What happens when things break

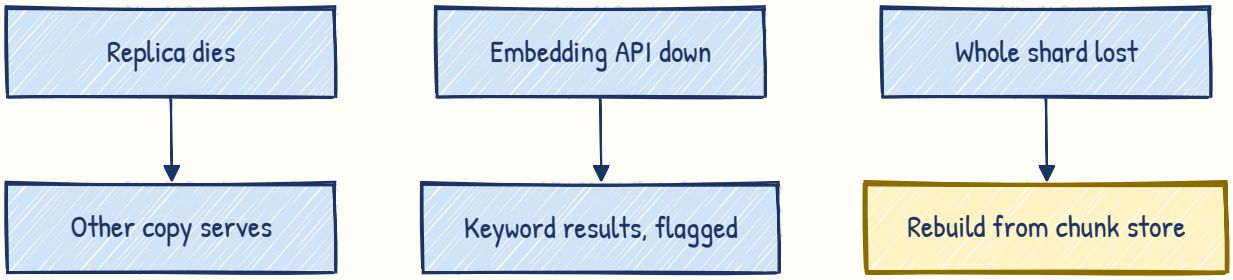


Figure 6. Failure containment. Each failure degrades one capability, and search keeps answering where it can, with a flag telling callers what was lost.

Failure	Containment	What users notice
A shard replica dies	Route to the other copy	Possibly slower
Embedding API down	Cached vectors and keyword results, with `degraded: true`	Weaker meaning-based results
Reranker slow	Return the fused order, with `degraded: true`	Slightly worse order
Catalog unavailable	Use cached routing, and block promotions	Nothing, for a while
A whole shard lost	Rebuild from the chunk store	Affected tenants degrade until it's done

9. Trade-offs

Decision	Choice	Why	Alternative
Tenancy	Tiered	Fits uneven tenant sizes	All shared (risky) or all separate (costly)
Isolation	Three layers	One layer can fail	Filter only
Compression	int8, plus full vectors on disk	Measured, reasonable recall	Binary, if a measurement supports it
Ingest	Async through Kafka	Absorbs bursts, retries safely	Synchronous, which ties upload to embedding
Fusion	RRF	No tuning, no scale issue	Weighted, if evals justify it
Store	Depends on AI04 L4's method	Depends on your constraints	pgvector at the lower end, a dedicated engine at this scale
Model version	Per tenant	Migrate gradually	Global switch, which is a big-bang risk

At this scale, I'd lean towards a dedicated engine with native tenant support, since 200 million chunks and 5,000 tenants test pgvector's partitioning approach. At 3 million chunks and 50 tenants, I'd choose pgvector. **State this condition out loud in an interview.**

10. Follow-up questions

Q A customer says, "Prove that our data can't be read by other tenants." What do you show?

A The three layers: token-derived routing, physical or row-level separation, and the automated cross-tenant test that runs on each release, with its history. Add audit logs of queries by tenant, and tenant-scoped caches. Dedicated infrastructure is available if contractually required.

Q Recall for one tenant fell from 0.97 to 0.82 last month, with no deployment. What happened?

A Probably data change: rapid growth, or new topics, causing drift, or too many tombstones. It might also have crossed an index setting's effective range, or begun spilling out of memory. Check recall by segment against exact search, the tenant's growth, and deleted-entry counts. Then rebuild its index, raise `ef_search`, or promote it.

Q How would you cut cost by half?

A First measure. Options: shorter dimensions if recall holds, stronger quantization if recall holds, moving cold tenants to disk-based or serverless storage, lowering replica counts for small tenants off-peak, and caching query embeddings. Compare each using recall at the same target, and don't trade away isolation.

Q A tenant's search must reflect permissions on individual documents inside that tenant. Does this design handle it?

A Only partly. Tenant isolation doesn't handle per-user rights. Store access groups as filter fields, apply them during search, and re-check the source for sensitive results, since rights can change after indexing. Avoid filtering by huge lists of user IDs, which Pinecone's documentation names as an anti-pattern. Access groups scale better. AI09 L4 goes further.

Certification lens

How this lesson maps to CCAR-F

Not tested directly. Infrastructure, deployment and vector database details are all out of scope. It's here for your production goal and senior interviews.

Revision - Lesson 5

→ Tenant sizes are very uneven: in this model, the largest is 2,500× the median, and 21 tenants hold 40% of data.

→ Tiered tenancy: shared storage for small tenants, dedicated indexes for large or regulated ones.

→ Isolation has three layers: token-based routing, storage separation, and automated cross-tenant tests.

→ Result caches must be tenant-scoped.

→ Sizing: 872 GB for float32 falls to 258 GB for int8. Don't assume binary works.

→ Routing to one target keeps shard searches at 2,000 per second, against 6,000 with fan-out to 3 shards.

→ Promotion and model migration both follow build beside, double-write, compare, then switch.

→ Store the model version per tenant, to migrate gradually.

→ Tombstone document deletes at once, and drop whole namespaces for tenants.

→ The durable chunk store makes every index rebuildable.

Your notes

Module Revision

Vector databases and search - on one page

Embedding models · A private coordinate system. Pin it, store its version with each vector, and refuse mismatches. Choose on your own data. Anthropic has no embedding model of its own.

Cost levers · Shorter dimensions and quantization. Each has a recall cost that varies by data, so measure it.

Recall · Your real quality metric. Compare with exact search on sampled queries, by segment. It decays silently.

Filters · Applied after the scan in pgvector by default. Correlated filters are far worse than the formula suggests, so partition tenants.

Hybrid · BM25 and vectors miss different queries. Merge by rank. Rerankers add precision on a shortlist, and can't recover what's missing.

Choosing a store · Start from constraints. Stay in Postgres until a measured limit. Keep the search store rebuildable. Benchmark at a fixed recall, with filters and writes.

Multi-tenancy · Tiered, with three isolation layers. Migrate and promote by building beside, then switching.

Self-assessment

Answer without looking back. Each has a specific answer in this module.

#	Question
1	Why does a model mismatch produce no error?
2	What should you choose an embedding model on?
3	What does `input_type` do?
4	What must you do after truncating a vector?
5	Name four rules for operating embeddings.
6	Why must a query cache key include the model version?
7	How do you measure recall in production?
8	Which HNSW settings can change without a rebuild?
9	What did 1 probe achieve in the lab, and why does that matter?
10	Why did a 10% tenant filter give 88% empty results?
11	Why did the average hide this?
12	Give five filtering strategies.
13	Compare half precision, int8 and binary.
14	Why did binary do badly in the lab, and what's the right conclusion?
15	Describe two-stage search.
16	Why did the BM25 and embeddings tie at 70% hide something important?
17	Why can't you add raw scores?
18	Why can't a reranker fix a recall problem?
19	When would you use MRR, and when recall@k?
20	What analyzer problem can break exact-code search?
21	State the decision method for choosing a store.
22	What signals say you should leave Postgres?
23	Why use `SET LOCAL` with a connection pool?

#	Question
24	What must a useful vector benchmark report?
25	Why treat the search store as a derived index?
26	Why does neither all-shared nor all-separate tenancy work at scale?
27	Name the three isolation layers.
28	Which caches are unsafe to share across tenants?
29	Walk through promoting a tenant with no downtime.
30	How do you delete a document from an HNSW-based system quickly?

Where this module is used

Idea	Where it returns
Model versioning and migration	AI05 L2, AI08 L5
Recall measurement	AI05 L6, AI10 L3
Hybrid search and rerank	AI05 L3–L4, AI08 L5
Tenant isolation	AI05 L7, AI09 L4, AI10 L6
pgvector design	AI08 L5
Derived index, CDC	AI05 L2, AI05 L7

Working space